Laboratorio Calificado 09

2026-1

ÍNDICE

1 Instrucciones 3
2 Contexto 3
3 Objetivos 4
4 Requisitos 4
5 Entregables 5
6 Rúbrica de Evaluación 5

Laboratorio Calificado 09

Simulación Gráfica de Trayectorias para Unidades Robóticas en una Línea de Producción Automatizada

1 Instrucciones

Se ha creado la tarea “Laboratorio Calificado 9” en la sección Laboratorio Calificado 9 de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con todos los archivos del desarrollo realizado en el laboratorio. El nombre del archivo comprimido deberá tener el formato:
L9_<Código del alumno de 8 dígitos>.<Extensión del archivo>
Como ejemplo, el nombre de archivo del alumno 20236969 sería “L9_20236969.zip”. Es COMPLETA RESPONSABILIDAD DE CADA ALUMNO el colocar nombres correctos a sus archivos y evitar así confusiones al momento de la calificación.
Está permitido usar cualquier material disponible en PAIDEIA y la documentación oficial de Microsoft. No está permitido el uso de materiales externos, IA o código de otros alumnos. De detectarse coincidencias, todos los implicados obtendrán la nota inapelable de cero.

2 Contexto

El Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica requiere asegurar la integridad física de los operarios y de la infraestructura de la planta. Para ello, cada manipulador robótico debe tener configurada una **Zona Límite de Seguridad Estructural (Workspace Limit)** que restrinja su movimiento máximo permisible.

Debido a la cinemática de los brazos robóticos planos, su alcance perimetral no forma un círculo perfecto, sino una geometría en forma de **Cardioide** debido a las restricciones mecánicas y de articulación de sus eslabones. El software de supervisión debe ser capaz de graficar este contorno de seguridad a partir de los parámetros de diseño del robot para validar visualmente que ninguna estación colisione con su rango de operación.

La ecuación polar que modela el radio máximo de alcance r en función del ángulo del motor θ (de 0 a 2π radianes) es:

$$r = a + b \cdot \cos(\theta)$$

Donde:

- $\boldsymbol{\theta}$ representa el ángulo de giro de la articulación base (rango de 0 a 2π radianes).
- **r** es el radio de alcance máximo resultante desde la base del robot (en milímetros).
- **a** representa la longitud del eslabón principal (parámetro de entrada en mm).
- **b** representa el factor de excentricidad o alcance del segundo eslabón (parámetro de entrada en mm).

(Nota: Si a = b, la curva es una cardioide pura con un punto de inflexión en el origen, simulando una zona muerta mecánica).

Para poder graficar esta zona en el plano cartesiano del monitor (X, Y), el alumno deberá transformar primero las coordenadas polares calculadas utilizando las identidades trigonométricas estándar:

$$x = r \cdot \cos(\theta)$$
$$y = r \cdot \sin(\theta)$$

3 Objetivos

Implementar una interfaz gráfica en C++/CLI que permita:

- Representar un plano de planta bidimensional (X-Y) centrado en la GUI utilizando las herramientas nativas de dibujo.
- Calcular y almacenar la curva límite de seguridad del robot usando ecuaciones cinemáticas polares.
- Introducir de manera dinámica los parámetros geométricos de los eslabones (a y b) desde la interfaz gráfica.
- Asociar perimetralmente el gráfico generado a una unidad robótica específica registrada.

4 Requisitos

a) Implementación del módulo matemático de la Cardioide (4 puntos)

- Definir una clase especializada (e.g., WorkspaceCalculator) encargada de realizar la discretización de la curva.
- Utilizar un bucle que varíe θ desde 0 hasta 2π (con un paso fino, por ejemplo de 0.05 radianes) para calcular los correspondientes valores de r, y posteriormente convertirlos a coordenadas cartesianas (x, y) reales en milímetros.
- Almacenar estos puntos cartesianos en estructuras dinámicas (List<PointF> o vectores similares).
- En C++/CLI, se utilizarán las funciones trigonométricas nativas de la librería:

```
double r = a + b * Math::Cos(theta);
double x_mm = r * Math::Cos(theta);
double y_mm = r * Math::Sin(theta);
```

b) Diseño de la GUI y mapeo de escala para dibujo (12 puntos)

- Implementar un PictureBox (se sugiere de $400 \times 400\ px$) para el lienzo.
- Dibujo de Ejes: A diferencia del laboratorio anterior, al ser una curva polar que se extiende en los 4 cuadrantes, los ejes coordenados de la planta (0,0) deben dibujarse justo en el centro del PictureBox (es decir, en la posición de píxeles x=200, y=200).
- Mapeo de Escala de mm a Píxeles: Dado que los valores de x_mm y y_mm pueden ser tanto positivos como negativos, la conversión para ubicar el píxel exacto en la pantalla respecto al centro (200, 200) será:
  - Escala horizontal (x): $px = 200 + (x_mm \times FactorEscala)$
  - Escala vertical (y): $py = 200 - (y_mm \times FactorEscala)$ (El signo menos invierte el eje vertical para que los valores positivos de Y vayan hacia arriba de la pantalla).
- Incluir los controles de ingreso para los milímetros de los eslabones a y b, el botón "Graficar Límite de Seguridad" y "Limpiar".
- El botón de graficar debe recorrer la colección de puntos uniendo las coordenadas píxel por píxel usando Graphics->DrawLine o Graphics->DrawLines.

c) Ejecución, validación y pruebas (4 puntos)

Simular y visualizar en la pantalla al menos dos configuraciones de alcance:

1. Una Cardioide pura ($a = b = 100mm$).
2. Un Limaco con lazo interno ($a = 50mm, b = 100mm$), que modela físicamente una zona donde el brazo se traslapa sobre su propio cuerpo.

Adjuntar en el informe final el código fuente desarrollado en capas y las capturas de pantalla de las complejas formas geométricas generadas en el entorno gráfico.

5 Entregables

- Código fuente completo en C++/CLI, bien comentado.
- Capturas de pantalla del sistema funcionando.

6 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente (100%)	Bueno (75%)	Regular (50%)	Deficiente (25%)	No Cumple (0%)
a) Implementación del módulo de gráfico (4 pts)	Calcula correctamente los pares (x, y) para un rango definido, usando estructuras de datos adecuadas y métodos modulares. (4 pts)	Cálculo correcto con estructura mejorable. (3 pts)	Cálculo parcial o con errores menores. (2 pts)	Cálculo incompleto o erróneo. (1 pt)	No se implementa el módulo. (0 pts)
b) Diseño de la GUI para graficar (12 pts)	GUI completa y funcional con plano cartesiano, controles para k y x_0, botones operativos y asociación con máquina. (10-12 pts)	GUI funcional con errores menores en interacción o escala. (7-9 pts)	GUI con funcionalidades limitadas o visualización imperfecta. (4-6 pts)	GUI incompleta o con fallas en la representación gráfica. (1-3 pts)	No se implementa la GUI o no permite graficar. (0 pts)
c) Ejecución, validación y pruebas (4 pts)	Se prueba con una máquina real, incluye varias capturas con diferentes parámetros y código bien documentado. (4 pts)	Pruebas funcionales con documentación básica. (3 pts)	Pruebas parciales o poco documentadas. (2 pts)	Pruebas incompletas o sin evidencia suficiente. (1 pt)	No se realizan pruebas ni documentación. (0 pts)